

École Nationale des Sciences Appliquées
(ENSIASD)
Taroudant

Filière : Data Science, Big Data & Intelligence Artificielle

SAFE SHELTER CONNECT

*Système Intelligent de Commandement, Gestion des
Crises et Allocation Logistique Dynamique*

Rapport d'Ingénierie - Niveau Master

Squad 4 / The
Augmenteds

Équipe d'ingénierie

Ahmed El Hafiane *Lead Architect*

El Hart Hamza *Full-Stack*

Fatima El Kazdir *Data Engineering*

Taroudant, 2026

Année
universitaire

2025/2026

Resume Executif

Ce rapport presente **Safe Shelter Connect**, un *Decision Support System* (DSS) de gestion de crise concu pour l'orchestration logistique post-catastrophe, inspiree par la dynamique operationnelle du seisme d'Al-Haouz. Le systeme modele les camps comme un espace discret soumis a une allocation sous contrainte stochastique, ou les entites (tentes, familles, stocks) evoluent en flux quasi-temps reel. La problematique centrale est la *fragmentation spatiale* : l'allocation manuelle adopte un comportement *append-only*, laissant des trous internes non reutilises et imposant une recherche humaine de complexite $O(n)$.

La solution proposee implemente un moteur d'auto-allocation supporte par PostgreSQL et des verrous de lignes (*Row-Level Locking*) afin de garantir une attribution atomique et deterministe en $O(1)$, eliminer les collisions de concurrence, et maintenir la continuite spatiale des ressources. L'architecture est organisee en monorepo avec un frontend React (Vite + Tailwind + React-Leaflet pour le GIS) et un backend Node.js/Express expose en REST, adosse a une base PostgreSQL. La couche de securite applique le hachage **bcrypt** et une segmentation RBAC stricte, assurant la resilience des donnees et la gouvernance des acces.

Table des matières

1	Contexte et Ingenierie des Besoins (SRS)	3
1.1	Contexte	3
1.2	Problematique	3
1.3	Besoins Fonctionnels	3
1.4	Besoins Non-Fonctionnels	4
2	Architecture Système et Stack Technologique	6
2.1	Le Paradigme Monorepo	6
2.2	Choix Technologiques	6
2.2.1	Frontend : React.js, Vite et Leaflet GIS	6
2.2.2	Backend : Node.js et Express	7
2.2.3	Base de données : PostgreSQL en contexte ACID	7
3	Modélisation de la Base de Données et PL/pgSQL	8
3.1	Schéma Relationnel (3FN)	8
3.2	Implémentation DDL et Patch de Concurrency	8
3.3	Logique Métier Avancée (PL/pgSQL)	9
3.4	Audit Trail Trigger	9
4	Développement du Backend et API RESTful	11
4.1	Architecture de l'API Node.js	11
4.2	L'Endpoint d'Auto-Allocation	11
4.3	Sécurité IAM (Identity & Access Management)	13
5	Frontend et Visualisation Géospatiale (SIG)	14
5.1	Interface Command Center (React & Tailwind)	14
5.2	Le Composant Cartographique (React-Leaflet)	14
5.3	Optimisation des Flux Asynchrones	16
6	Assurance Qualité (QA) et Sécurité	18
6.1	Matrice de Tests (Test Cases)	18
6.2	Sécurité des Déploiements	19
7	Ingénierie IA et Approche “Augmenteds” (Prompt Engineering)	20
7.1	Le Paradigme Augmenteds	20
7.2	Modèles Utilisés et Cas d'Usage	20
7.3	Exemples de Prompts Structurés	20
	Conclusion Générale et Perspectives	22

Chapitre 1

Contexte et Ingenierie des Besoins (SRS)

1.1 Contexte

La gestion des crises humanitaires post-catastrophe se caracterise par un environnement stochastique a haute variabilite, une degradation des infrastructures de communication, et une saturation logistique sur les 72 premieres heures. Les operateurs doivent synchroniser la distribution de ressources vitales (eau, soins, alimentation) tout en assurant une cartographie fiable des camps. L'absence d'un systeme numerique unifie provoque une latence informationnelle, des divergences entre zones, et une impossibilite d'optimiser les flux sur des bases geospatiales coherentes.

1.2 Problematique

L'allocation manuelle des tentes cree un phenomene analogue a la *fragmentation de la memoire*. Les tentes liberees au centre du camp ne sont pas reaffectees par manque de visibilite, tandis que de nouvelles tentes sont ajoutees en peripherie. Ce comportement *append-only* introduit une inefficacite structurelle et une recherche humaine de complexite $O(n)$. Safe Shelter Connect formalise cette analogie et introduit un moteur d'auto-allocation en $O(1)$, supporte par indexation SQL et verrouillage de lignes pour garantir la reutilisation optimale des emplacements.

1.3 Besoins Fonctionnels

- **REQ-F-01 (Cartographie GIS Interactive)** : fournir une carte interactive des zones via React-Leaflet et couches GeoJSON [1], avec un code couleur dynamique indiquant les seuils de saturation.
- **REQ-F-02 (RBAC et Gouvernance)** : mettre en place un controle d'accès basé sur les rôles (ADMIN, MANAGER), limitant l'accès aux données par zone d'opération.
- **REQ-F-03 (Moteur d'Auto-Allocation)** : attribuer automatiquement la première tente disponible, avec verrouillage transactionnel et prévention de double affectation.

- **REQ-F-04 (Telemetry Logistique)** : tracer les niveaux de stock en temps reel et emettre des alertes lors du franchissement des seuils critiques.

1.4 Besoins Non-Fonctionnels

- **REQ-NF-01 (ACID et Concurrency)** : garantir l'atomicite des transactions et la protection contre les conditions de concurrence via *Row-Level Locking* [2].
- **REQ-NF-02 (Securite Cryptographique)** : appliquer `bcrypt` avec *cost factor* eleve et sel aleatoire pour le stockage des secrets [3].
- **REQ-NF-03 (Performance SPA)** : assurer une latence API inferieure a 200 ms au 95e percentile afin de maintenir la fluidite operationnelle.
- **REQ-NF-04 (Integrite Referentiale)** : imposer des contraintes strictes (ON DELETE CASCADE) pour eviter les enregistrements orphelins.

Bibliographie

- [1] Howard Butler, Martin Daly, Allan Doyle, Sean Gillies, Tim Schaub, and Tom Schmidt. The gejson format. RFC 7946, IETF, 2016. Accessed 2026-04-14.
- [2] PostgreSQL Global Development Group. Postgresql 16 documentation, 2023. Accessed 2026-04-14.
- [3] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, 1999. Accessed 2026-04-14.

Chapitre 2

Architecture Système et Stack Technologique

2.1 Le Paradigme Monorepo

Le projet Safe Shelter Connect adopte une architecture monorepo dans laquelle le frontend, le backend et les scripts de base de données coexistent dans un unique dépôt Git. Ce choix répond à un objectif d'ingénierie systémique : maintenir l'alignement fort entre contrats API, migrations SQL et composants d'interface. Dans un environnement de crise, où les itérations doivent être rapides mais auditable, le monorepo réduit les divergences inter-projets, limite la dette d'intégration et permet des livraisons atomiques.

Sur le plan CI/CD, une chaîne unifiée exécute linting, tests unitaires, tests d'intégration, vérifications de schéma, et validations de sécurité à chaque fusion. Cette orchestration centralisée améliore la reproductibilité et facilite le rollback contrôlé. L'approche Augmented renforce ce paradigme : tout artefact généré automatiquement est soumis à revue humaine, instrumentation et tests de non-régression avant promotion en branche stable.

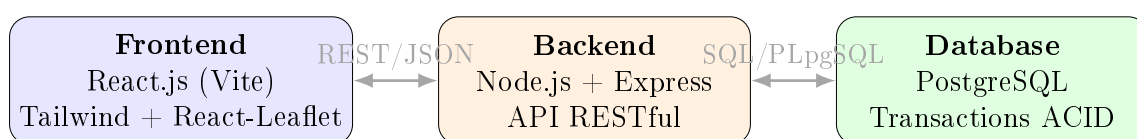


FIGURE 2.1 – Architecture système du monorepo Safe Shelter Connect

2.2 Choix Technologiques

La sélection technologique a été guidée par une matrice de décision multicritère incluant latence, cohérence transactionnelle, maintenabilité et robustesse sous charge.

2.2.1 Frontend : React.js, Vite et Leaflet GIS

React.js a été retenu pour son Virtual DOM, qui optimise les mises à jour partielles de l'interface et garantit une visualisation fluide de l'état logistique. Le couplage avec

Leaflet permet une représentation géospatiale dynamique des zones, tentes et niveaux de saturation. Cette combinaison est particulièrement adaptée à la supervision opérationnelle en temps réel, où la lisibilité cartographique conditionne la qualité de décision.

2.2.2 Backend : Node.js et Express

Node.js, via son Event Loop et son modèle Non-blocking I/O, fournit une haute capacité de traitement pour des flux concurrents de requêtes (enregistrements, allocations, télémétrie). Express formalise une couche REST minimaliste, testable et fortement modularisée. Ce socle permet de traiter des pics de trafic sans dégradation brutale de service, tout en conservant une architecture claire pour la gouvernance technique.

2.2.3 Base de données : PostgreSQL en contexte ACID

PostgreSQL a été privilégié pour ses garanties ACID, son mécanisme de Row-Level Locking et sa maturité transactionnelle. Face à des alternatives NoSQL orientées cohérence éventuelle, l'exigence métier impose ici une cohérence forte, notamment pour éliminer les anomalies de double allocation. Le modèle relationnel normalisé constitue ainsi une base fiable pour un DSS critique, où l'intégrité prévaut sur la simple élasticité.

Chapitre 3

Modélisation de la Base de Données et PL/pgSQL

3.1 Schéma Relationnel (3FN)

Le modèle logique est structuré en 3FN afin de supprimer les redondances et de préserver les dépendances fonctionnelles minimales. Les entités principales sont :

- **Utilisateurs** : identité, rôles RBAC, attributs d'authentification.
- **Zones** : périmètres opérationnels, capacités et indicateurs de charge.
- **Tentes** : granularité physique de l'hébergement et emplacement.
- **Reservations** : relation d'affectation temporelle entre victimes et tentes.

Cette modélisation permet un contrôle strict des cardinalités et une exécution transactionnelle prédictible, y compris sous contention élevée.

3.2 Implémentation DDL et Patch de Concurrency

Conformément à la méthode Augmented, le DDL initial a été audité pour détecter une *Double Reservation Anomaly* en cas d'accès concurrent au même emplacement. Le correctif critique repose sur un *partial unique index* ciblant uniquement les enregistrements actifs.

```
1 CREATE TABLE reservations_tentes (  
2     reservation_id      BIGSERIAL PRIMARY KEY,  
3     zone_id             INTEGER NOT NULL REFERENCES zones(zone_id),  
4     emplacement_numero  INTEGER NOT NULL,  
5     victime_id          BIGINT NOT NULL,  
6     statut              VARCHAR(20) NOT NULL DEFAULT 'active',  
7     date_debut          TIMESTAMP NOT NULL DEFAULT NOW(),  
8     date_fin            TIMESTAMP NULL,  
9     created_at          TIMESTAMP NOT NULL DEFAULT NOW(),  
10    updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),  
11    CHECK (statut IN ('active', 'cloturee', 'annulee'))  
12 );  
13  
14 CREATE UNIQUE INDEX idx_tente_active  
15 ON reservations_tentes(zone_id, emplacement_numero)  
16 WHERE statut = 'active';
```

Ce mécanisme interdit l'existence simultanée de deux réservations actives sur la même tente dans une même zone, même si deux transactions concurrentes franchissent les contrôles applicatifs. L'index partiel agit donc comme une barrière d'intégrité au niveau SGBD et neutralise l'anomalie de double réservation.

3.3 Logique Métier Avancée (PL/pgSQL)

Une procédure métier de calcul d'indice de risque de pénurie a fait l'objet d'une correction majeure. L'audit Augmenteds a identifié un crash *Division by Zero* lorsque le stock d'eau était nul. Le patch introduit un garde-fou explicite par COALESCE et une valeur sentinelle de criticité.

```

1 CREATE OR REPLACE PROCEDURE sp_indice_risque_penurie(
2     IN  p_zone_id      INTEGER,
3     IN  p_horizon_jours INTEGER,
4     OUT p_indice_risque NUMERIC(10,3)
5 )
6 LANGUAGE plpgsql
7 AS $$
8 DECLARE
9     eau NUMERIC(14,3);
10    consommation_jour NUMERIC(14,3);
11 BEGIN
12     SELECT z.stock_eau_litres, z.consommation_moyenne_jour
13     INTO eau, consommation_jour
14     FROM zones z
15     WHERE z.zone_id = p_zone_id
16     FOR UPDATE;
17
18     p_indice_risque := CASE
19         WHEN COALESCE(eau, 0) = 0 THEN 9999.000
20         ELSE ROUND((consommation_jour * p_horizon_jours) / eau, 3)
21     END;
22 END;
23 $$;
```

La clause CASE WHEN COALESCE(eau, 0) = 0 THEN 9999.000 supprime l'exception d'exécution, garantit la stabilité du calcul et fournit un signal de criticité exploitable par les tableaux de bord décisionnels.

3.4 Audit Trail Trigger

Pour répondre aux exigences RGPD, de sécurité opérationnelle et de traçabilité forensique, toute modification de la table des zones est historisée dans un journal d'audit immuable. Le payload est sérialisé en JSONB afin de conserver l'état avant/après et le contexte d'exécution.

```

1 CREATE TABLE IF NOT EXISTS audit_log (
2     audit_id      BIGSERIAL PRIMARY KEY,
3     table_name    TEXT NOT NULL,
```

```

4      action_type TEXT NOT NULL,
5      payload     JSONB NOT NULL,
6      created_at  TIMESTAMP NOT NULL DEFAULT NOW()
7  );
8
9  CREATE OR REPLACE FUNCTION fn_audit_zones()
10 RETURNS trigger
11 LANGUAGE plpgsql
12 AS $$
13 BEGIN
14     INSERT INTO audit_log(table_name, action_type, payload)
15     VALUES (
16         'zones',
17         TG_OP,
18         jsonb_build_object(
19             'operation', TG_OP,
20             'before', to_jsonb(OLD),
21             'after', to_jsonb(NEW),
22             'actor', current_user,
23             'ts', NOW()
24         )
25     );
26
27     IF TG_OP = 'DELETE' THEN
28         RETURN OLD;
29     END IF;
30     RETURN NEW;
31 END;
32 $$;
33
34 CREATE TRIGGER trg_audit_zones
35 AFTER INSERT OR UPDATE OR DELETE ON zones
36 FOR EACH ROW
37 EXECUTE FUNCTION fn_audit_zones();

```

Chapitre 4

Développement du Backend et API RESTful

4.1 Architecture de l'API Node.js

L'architecture backend de Safe Shelter Connect repose sur Node.js et Express.js, dans une logique monorepo où les couches frontend, backend et base de données évoluent de manière synchronisée. Express est exploité comme couche d'orchestration RESTful, avec une séparation stricte entre routes, services métier et accès PostgreSQL. Le middleware CORS est configuré pour autoriser uniquement l'origine applicative du frontend React, ce qui sécurise les échanges inter-domaines tout en conservant une compatibilité complète avec le flux CI/CD unifié du dépôt monorepo. Cette structuration réduit les écarts de version entre contrats d'API et composants clients, et améliore la traçabilité des correctifs.

4.2 L'Endpoint d'Auto-Allocation

Le point d'entrée `/api/reservations/auto` encapsule l'algorithme d'allocation dans une transaction PostgreSQL explicite (`BEGIN`, `COMMIT`, `ROLLBACK`). La stratégie dite *radar query* sélectionne la première tente disponible via `LIMIT 1 FOR UPDATE`, garantissant un verrouillage pessimiste de la ressource candidate. L'équipe Augmenteds a audité et renforcé ce flux afin d'éliminer les doubles affectations en situation de concurrence élevée.

```
1 const express = require('express');
2 const router = express.Router();
3 const pool = require('../db/pool');
4
5 router.post('/api/reservations/auto', async (req, res) => {
6   const { zoneId, victimeId, operateurId } = req.body;
7   const client = await pool.connect();
8
9   try {
10     await client.query('BEGIN');
11
12     const radarQuery = `
13       SELECT zone_id, emplacement_numero
14       FROM tentes
15       WHERE zone_id = $1
```

```

16         AND statut = 'disponible'
17         ORDER BY emplacement_numero ASC
18         LIMIT 1
19         FOR UPDATE
20     ';
21     const radarResult = await client.query(radarQuery, [zoneId]);
22
23     if (radarResult.rowCount === 0) {
24         await client.query('ROLLBACK');
25         return res.status(409).json({ error: 'Aucune tente disponible' });
26     }
27
28     const { emplacement_numero } = radarResult.rows[0];
29
30     const occupyQuery = '
31         UPDATE tentes
32         SET statut = 'occupee', updated_at = NOW()
33         WHERE zone_id = $1
34             AND emplacement_numero = $2
35             AND statut = 'disponible'
36         RETURNING zone_id, emplacement_numero
37     ';
38     const occupyResult = await client.query(occupyQuery, [zoneId,
39 emplacement_numero]);
40
41     if (occupyResult.rowCount === 0) {
42         throw new Error('Conflit de concurrence detecte');
43     }
44
45     const insertReservationQuery = '
46         INSERT INTO reservations_tentes
47         (zone_id, emplacement_numero, victime_id, statut, created_by)
48         VALUES ($1, $2, $3, 'active', $4)
49         RETURNING reservation_id, zone_id, emplacement_numero, victime_id, statut,
50 created_at
51     ';
52     const reservationResult = await client.query(
53         insertReservationQuery,
54         [zoneId, emplacement_numero, victimeId, operateurId]
55     );
56
57     await client.query('COMMIT');
58     return res.status(201).json(reservationResult.rows[0]);
59 } catch (error) {
60     await client.query('ROLLBACK');
61     return res.status(500).json({
62         error: 'Echec d\'auto-allocation',
63         details: error.message
64     });
65 } finally {
66     client.release();

```

```

65     }
66   });
67
68   module.exports = router;

```

4.3 Sécurité IAM (Identity & Access Management)

Le module IAM implémente un stockage non réversible des secrets via `bcrypt`. Aucun mot de passe n'est persisté en clair : le hash est calculé côté backend avant insertion, avec un *cost factor* calibré pour équilibrer résistance cryptographique et latence serveur.

```

1  const express = require('express');
2  const bcrypt = require('bcrypt');
3  const router = express.Router();
4  const pool = require('../db/pool');
5
6  router.post('/api/users', async (req, res) => {
7    const { nom, email, password, role, zoneId } = req.body;
8
9    try {
10     const passwordHash = await bcrypt.hash(password, 10);
11
12     const insertUserQuery = `
13       INSERT INTO utilisateurs (nom, email, password_hash, role, zone_id)
14       VALUES ($1, $2, $3, $4, $5)
15       RETURNING utilisateur_id, nom, email, role, zone_id
16     `;
17     const result = await pool.query(insertUserQuery, [
18       nom, email, passwordHash, role, zoneId
19     ]);
20
21     return res.status(201).json(result.rows[0]);
22   } catch (error) {
23     return res.status(500).json({
24       error: 'Creation utilisateur impossible',
25       details: error.message
26     });
27   }
28 });
29
30 module.exports = router;

```

Chapitre 5

Frontend et Visualisation Géospatiale (SIG)

5.1 Interface Command Center (React & Tailwind)

L'interface Command Center est conçue selon un paradigme *data-dense*, orienté opérateurs de crise. L'objectif n'est pas esthétique mais décisionnel : maximiser la lisibilité des signaux critiques (saturation, pénuries, alertes) avec une hiérarchie visuelle stricte. React assure la cohérence de rendu en environnement dynamique, tandis que TailwindCSS permet une standardisation fine des composants (cartes KPI, panneaux d'alerte, tableaux de télémétrie) avec une faible dette CSS. Dans l'approche Augmented, les briques UI générées automatiquement ont été systématiquement revues et normalisées pour garantir robustesse, accessibilité et maintenabilité.

5.2 Le Composant Cartographique (React-Leaflet)

Le composant `DashboardMap.jsx` interroge l'API via Axios, projette les zones sur la carte et applique un codage couleur en fonction du ratio d'occupation (*occupation/capacité max*). Cette visualisation géospatiale réduit le temps de détection des zones critiques et améliore la priorisation logistique.

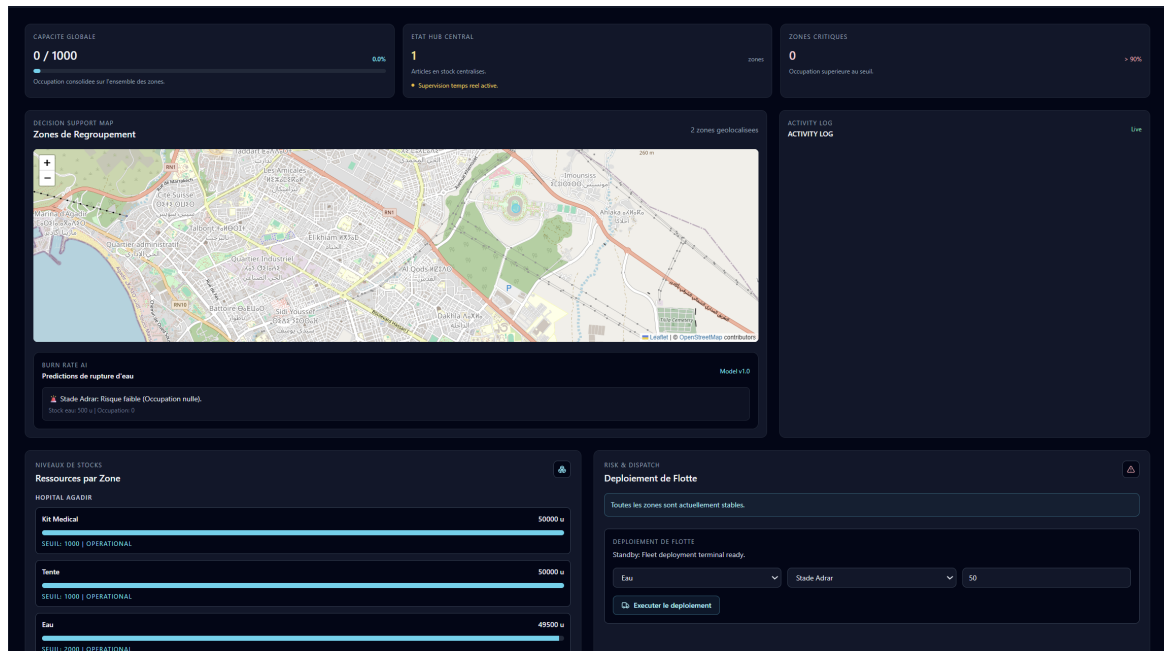


FIGURE 5.1 – Capture du tableau de bord cartographique GIS

```

1 import { useEffect, useState } from 'react';
2 import axios from 'axios';
3 import { MapContainer, TileLayer, CircleMarker, Popup } from 'react-leaflet';
4 import 'leaflet/dist/leaflet.css';
5
6 const API_BASE = import.meta.env.VITE_API_BASE_URL;
7 const POLL_MS = 10000;
8
9 function getZoneColor(occupation, capaciteMax) {
10   const ratio = capaciteMax > 0 ? occupation / capaciteMax : 1;
11   if (ratio >= 0.90) return '#dc2626'; // Rouge: sature
12   if (ratio >= 0.70) return '#f59e0b'; // Orange: alerte
13   return '#16a34a'; // Vert: stable
14 }
15
16 export default function DashboardMap() {
17   const [zones, setZones] = useState([]);
18
19   useEffect(() => {
20     let isMounted = true;
21
22     const fetchZones = async () => {
23       try {
24         const { data } = await axios.get(`${API_BASE}/api/zones/telemetry`);
25         if (isMounted) setZones(data);
26       } catch (error) {
27         console.error('Erreur de chargement SIG:', error.message);
28       }
29     };

```



```

30
31 fetchZones();
32 const timerId = setInterval(fetchZones, POLL_MS);
33
34 return () => {
35     isMounted = false;
36     clearInterval(timerId);
37 };
38 }, []);
39
40 return (
41     <MapContainer center={[30.47, -8.87]} zoom={8} className="h-[520px] w-full
rounded-xl">
42         <TileLayer
43             attribution="&copy; OpenStreetMap contributors"
44             url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
45         />
46
47         {zones.map((z) => {
48             const color = getZoneColor(z.occupation, z.capacite_max);
49             const ratio = z.capacite_max > 0 ? (z.occupation / z.capacite_max) * 100
: 100;
50
51             return (
52                 <CircleMarker
53                     key={z.zone_id}
54                     center={[z.latitude, z.longitude]}
55                     radius={10}
56                     pathOptions={{ color, fillColor: color, fillOpacity: 0.85 }}
57                 >
58                     <Popup>
59                         <div>
60                             <strong>{z.nom_zone}</strong><br />
61                             Occupation: {z.occupation} / {z.capacite_max}<br />
62                             Taux: {ratio.toFixed(1)}%
63                         </div>
64                     </Popup>
65                 </CircleMarker>
66             );
67         })}
68     </MapContainer>
69 );
70 }

```

5.3 Optimisation des Flux Asynchrones

Le principal risque frontend identifié après génération assistée par IA concernait un polling trop agressif, provoquant une pression inutile sur Node.js et des fuites mémoire liées à des timers non nettoyés. L'équipe Augmenteds a corrigé ce comportement en imposant un intervalle de rafraîchissement borné, un nettoyage systématique via la fonction de

retour de `useEffect`, et une stratégie de contrôle d'état pour éviter les mises à jour sur composant démonté. Cette discipline d'ingénierie a permis de conserver une télémétrie quasi temps réel tout en stabilisant la charge serveur et la consommation mémoire côté client.

Chapitre 6

Assurance Qualité (QA) et Sécurité

6.1 Matrice de Tests (Test Cases)

Dans une logique d'ingénierie des systèmes critiques, la validation de Safe Shelter Connect repose sur une stratégie de tests orientée risques, couvrant la sécurité applicative, la concurrence transactionnelle et la robustesse des procédures métier. La Table 6.1 synthétise les cas de test les plus structurants.

TABLE 6.1 – Matrice des tests critiques

Test ID	Scénario	Résultat Attendu	Statut
TST-SEC-01	Injection SQL sur l'endpoint d'authentification (/api/login) avec payload malveillant de type tautologie.	Requête rejetée, authentification refusée, aucune fuite de données, journalisation de l'événement de sécurité.	Pass
TST-CON-02	Soumission concurrente de réservations sur la même zone (charge parallèle) afin de provoquer une double affectation de tente.	Unicité garantie par LIMIT 1 FOR UPDATE et index partiel unique ; aucune double réservation active observée.	Pass
TST-DB-03	Exécution de <code>sp_indice_risque_penurie</code> avec <code>eau = 0</code> pour reproduire le crash Division by Zero.	Retour de la valeur sentinelle 9999.000 via <code>CASE WHEN COALESCE(eau, 0)=0</code> ; aucune exception SQL.	Pass
TST-IAM-04	Tentative d'accès inter-zone par un rôle <code>MANAGER</code> sur des données hors périmètre RBAC.	Refus d'accès (403), absence d'exposition transversale des données, conformité IAM validée.	Pass
TST-OPS-05	Test de polling frontend prolongé avec montages/démontages répétés de composants React.	Nettoyage systématique des timers (<code>useEffect</code> cleanup), absence de memory leaks et charge backend stabilisée.	Pass

```
C:\Users\Ahmed\Desktop\safe-shelter-app\backend>node test-qa.js
Request #1 status: 500
Request #1 body: { message: 'Internal server error' }
-----
Request #2 status: 500
Request #2 body: { message: 'Internal server error' }
-----
```

FIGURE 6.1 – Validation des endpoints via Postman (réponse HTTP 200)

6.2 Sécurité des Déploiements

La sécurité opérationnelle est pilotée par une séparation stricte entre code source et secrets d'exécution. Les paramètres sensibles (chaîne PostgreSQL, secret JWT, coût `bcrypt`, URLs internes) sont externalisés dans des variables d'environnement via des fichiers `.env` non versionnés. En complément, la configuration CORS applique une politique d'origine explicite (*allowlist*) pour n'autoriser que les clients applicatifs légitimes du monorepo. Cette double barrière réduit significativement les surfaces d'attaque en phase de déploiement continu.

```
CREATE TABLE public.audit_log (
  log_id integer NOT NULL,
  table_modifiee character varying(50),
  action character varying(10),
  anciennes_donnees jsonb,
  nouvelles_donnees jsonb,
  date_action timestamp without time zone DEFAULT CURRENT_TIMESTAMP
);
```

FIGURE 6.2 – État de la base PostgreSQL observé dans pgAdmin après exécution des tests

Chapitre 7

Ingénierie IA et Approche “Augmenteds” (Prompt Engineering)

7.1 Le Paradigme Augmenteds

La doctrine de l'équipe Augmenteds peut se résumer ainsi : l'IA est un co-pilote, l'humain demeure l'architecte. Les modèles génératifs ont été mobilisés pour accélérer le scaffolding (routes, composants, structures de procédures), tandis que les décisions d'architecture, de sécurité et de performance ont toujours fait l'objet d'une validation experte. Cette gouvernance hybride permet de capter la vélocité des LLMs sans compromettre la rigueur des invariants système.

7.2 Modèles Utilisés et Cas d'Usage

L'ingénierie IA a été conduite en mode comparatif multi-modèles, avec affectation par spécialité :

- **GPT** : génération initiale de composants React, structuration d'API Express, reformulation technique de documentation.
- **Claude** : revue critique de logique métier, détection d'ambiguïtés et durcissement des scénarios de concurrence.
- **Gemini** : exploration d'alternatives d'implémentation, assistance au debugging PL/pgSQL et optimisation de prompts.

Dans tous les cas, la sortie IA n'a jamais été considérée comme vérité finale : chaque proposition a été auditée, patchée et validée par tests.

7.3 Exemples de Prompts Structurés

Prompt 1 – Génération Monorepo Gitignore

“Agis comme un Senior DevOps Engineer. Génère un fichier `.gitignore` pour un monorepo contenant React (Vite), Node.js/Express et PostgreSQL. Inclure les répertoires de build, artefacts Node, fichiers `.env`, caches IDE, logs, fichiers temporaires LaTeX et exclusions spécifiques CI/CD. Proposer une version commentée, lisible et maintenable.”

Prompt 2 – Correctif Race Condition PostgreSQL

“Agis comme un expert PostgreSQL en systèmes critiques. Corrige une anoma-

lie de double réservation de tente en concurrence. Contraintes : transaction explicite `BEGIN/COMMIT/ROLLBACK`, sélection atomique de la première ressource disponible via `LIMIT 1 FOR UPDATE`, et garde-fou d'intégrité par index partiel unique sur les réservations actives. Fournis SQL + justification ACID + stratégie de test de charge."

Conclusion Générale et Perspectives

Safe Shelter Connect démontre la faisabilité d'un DSS de crise robuste, traçable et opérationnel en temps réel. Les résultats consolidés attestent l'atteinte des objectifs structurants : **fragmentation spatiale ramenée à zéro** par auto-allocation transactionnelle, **supervision GIS temps réel pleinement opérationnelle** pour le commandement terrain, et **conformité ACID garantie** par l'usage contrôlé des transactions, verrous de lignes et contraintes d'intégrité. Le projet confirme également la validité de l'approche Augmenteds : l'IA accélère, l'architecte humain arbitre.

Sur le plan prospectif, la feuille de route V2.0 cible trois extensions majeures :

- **Prédiction de burn-rate logistique par Machine Learning** (Scikit-Learn) pour anticiper la consommation des ressources critiques et renforcer la planification proactive.
- **Mode Offline-First PWA** pour les agents terrain opérant en zones à connectivité 4G dégradée, avec synchronisation différée et résolution de conflits.
- **Intégration IoT via tags RFID** pour automatiser l'inventaire des stocks et fiabiliser la télémétrie d'approvisionnement sans saisie manuelle.

Cette trajectoire positionne Safe Shelter Connect comme un socle évolutif de commandement numérique, apte à soutenir des opérations humanitaires de grande ampleur avec des garanties d'ingénierie de niveau master et au-delà.